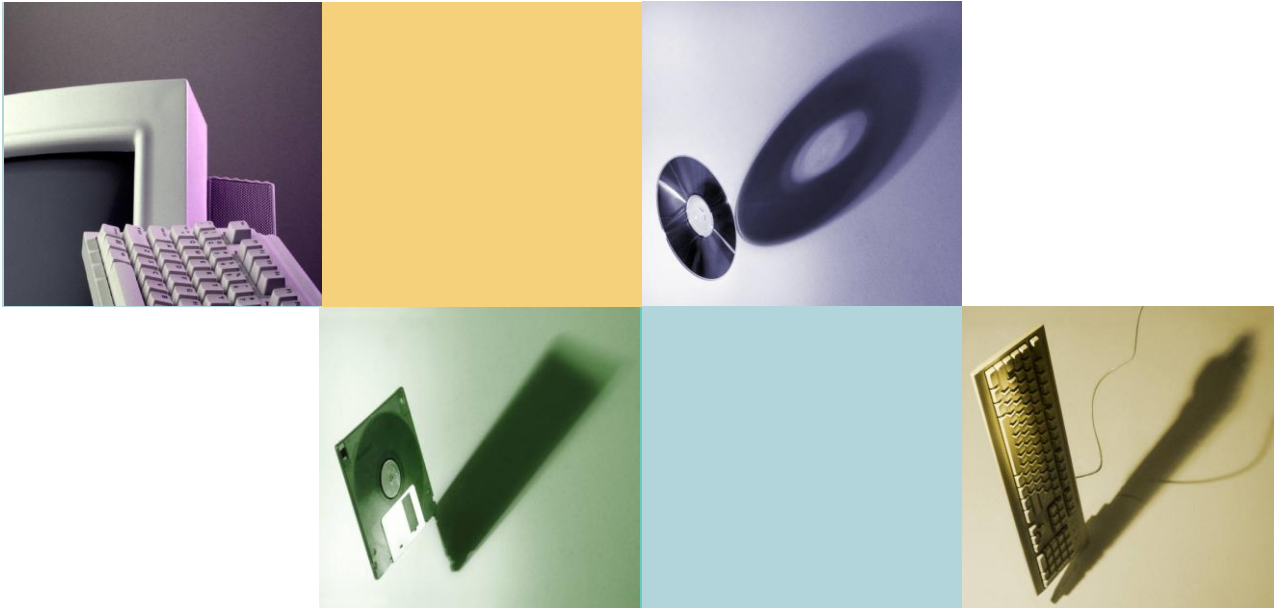


第2章 微处理器



第2章 微处理器

2.1 微处理器的编程结构

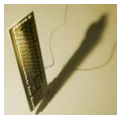
2.2 INTEL8086微处理器

2.3 Intel80X86及Pentium系列微处理器



第2章 微处理器

- 微处理器简称 μ P或MP (Microprocessor)
- 是微型计算机的中央处理器CPU(central process unit)
- 由运算器、控制器和寄存器组组成,
- 功能
 - ✓ 运算: 具有算术运算和逻辑运算功能,
 - ✓ 控制: 能够发出控制信号, 支配整个微机系统的工作, 是微型计算机的核心。



2.1 微处理器的编程结构

【主要内容】

微处理器在编程应用时所需的基本知识

- 微处理器的设计模型
- 工作模式
- 实地址模式下的分段寻址方式

【学习目标】

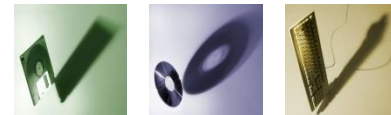
掌握微处理器的**可编程寄存器**，实模式下的**分段寻址方式**，**逻辑地址与物理地址**的概念及相互转换，了解微处理器的工作模式及其特点



2.1.1 微处理器的程序设计模型

程序可见 (program visible) 寄存器：8086及以上编程人员进行程序设计时，能够使用的寄存器。

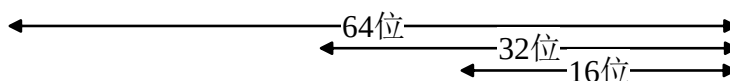
程序不可见寄存器 (program invisible) 80286及以上在应用程序设计中不能直接访问，但在系统程序设计中可以间接引用或通过特权指令访问



Intel 8086~Core2程序设计模型

64位寄存器名 32位寄存器名 16位寄存器名 8位寄存器名

RAX		EAX	AX	AH	AL
RBX		EBX	BX	BH	BL
RCX		ECX	CX	CH	CL
RDX		EDX	DX	DH	DL
RBP		EBP	BP		
RSI		ESI	SI		
RDI		EDI	DI		
RSP		ESP	SP		



R8				
R9				
R10				
R11				
R12				
R13				
R14				
R15				

RFLAGS		EFLAGS	FLAGS
--------	--	--------	-------

RIP		EIP	IP
-----	--	-----	----

CS
DS
ES
SS
FS
GS

寄存器位数

- 8086, 8088, 80286 — 16位
- 80386 ~ Core2 — 16位, 32位
- Pentium 4和Core2 — 含64位

- 通用寄存器
- 指令指针寄存器
- 标志寄存器
- 段寄存器



通用寄存器

64位寄存器名

32位寄存器名

16位寄存器名

8位寄存器名

RAX		EAX	AX	AH	AL
RBX		EBX	BX	BH	BL
RCX		ECX	CX	CH	CL
RDX		EDX	DX	DH	DL
RBP		EBP	BP		
RSI		ESI	SI		
RDI		EDI	DI		
RSP		ESP	SP		

累加器

基址寄存器

计数器

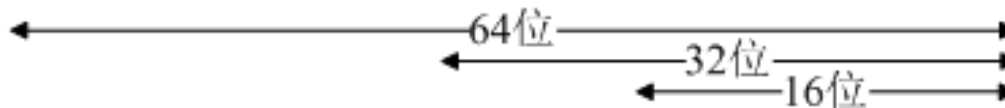
数据寄存器

基址指针

源变址寄存器

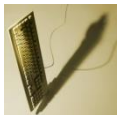
目标变址寄存器

堆栈指针



R8				
R9				
R10				
R11				
R12				
R13				
R14				
R15				

Pentium 4和Core2工作在扩展64位模式使用通用数据寄存器可按64位、32位、16位或8位寄存器寻址



通用寄存器

EAX(32-bits)	AX(16-bits)	AH, AL (8-bits)	accumulator
EBX	BX	BH,BL	base index
ECX	CX	CH,CL	counter
EDX	DX	DH,DL	data
EBP	BP		base pointer
ESP	SP		stack pointer
EDI	DI		destination index
ESI	SI		source index

EAX 累加器

ECX 计数器

EBP 基址指针

ESI 源变址寄存器

EBX 基址寄存器

EDX 数据寄存器

ESP 堆栈指针

EDI 目标变址寄存器



专用寄存器

64位

32位

16位

RFLAGS

EFLAGS

FLAGS

标志寄存器

RIP

EIP

IP

指令指针寄存器

CS

DS

ES

SS

FS

GS

段寄存器

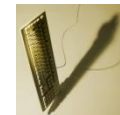
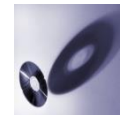
RIP/EIP (Instruction pointer) 指令指针寄存器

与代码段寄存器配合，寻址CPU要取出的下一条指令字节。

实模式 — IP (16位)

保护模式 — EIP (32位) 80386 及以上

64位模式 — RIP (64位)



专用寄存器

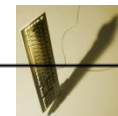
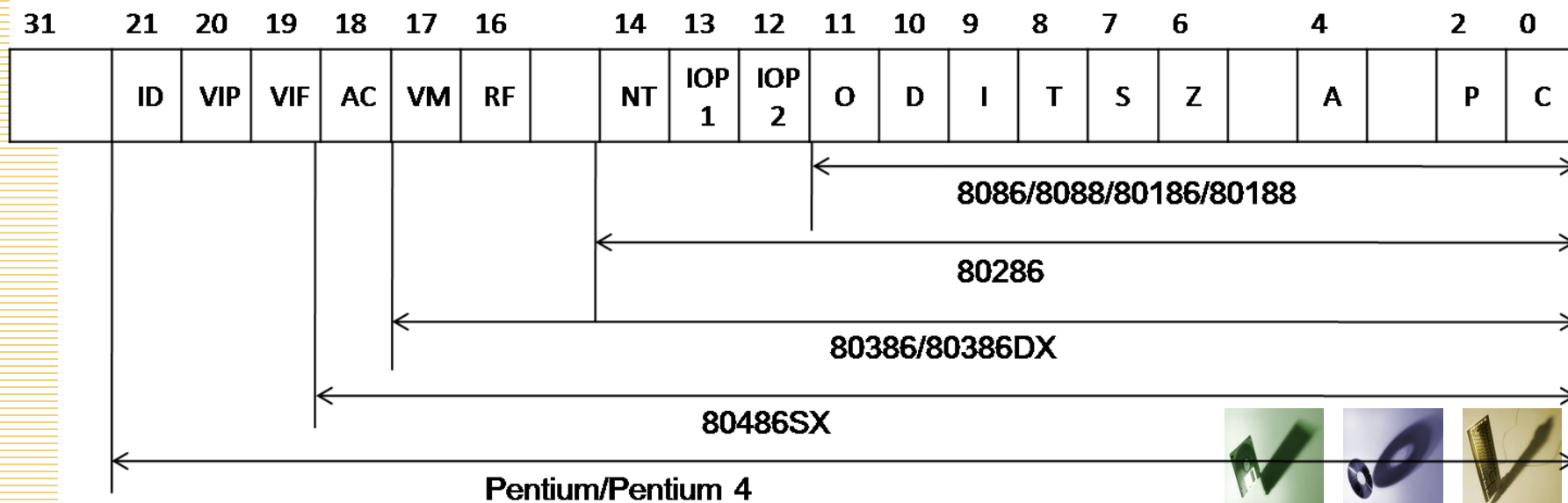
RFLAGS/EFLAGS 标志寄存器

指示微处理器的状态并控制它的操作

FLAGS(16-bits) 8086 — 80286

EFLAGS (32-bits) 80386 及以上

RFLAGS (64-bits)



段寄存器

保存段基址，与其他寄存器或数据配合，形成存储器地址

CS (code) 代码段寄存器

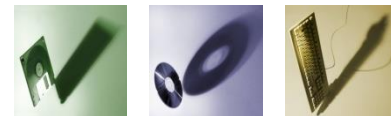
存放当前代码段的段基址。代码段是用于存放指令代码的存储区域，当前代码段是指CPU即时访问的代码段。

DS (data) 数据段寄存器

存放当前数据段的段基址。程序执行需要的数据经常存放于数据段中。

ES (extra) 附加段寄存器

存放当前附加段的段基址。附加段是一个附加的数据段，通常也用来存放数据，典型用法是在串操作指令中用来存放处理以后的数据。



段寄存器

SS (stack) 堆栈段寄存器

存放程序当前堆栈段的段基址。堆栈操作所处理的数据均存放于当前堆栈段中，由堆栈段寄存器**SS**和堆栈指针**SP**共同确定堆栈操作将要访问的存储单元地址。

段寄存器FS和GS

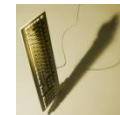
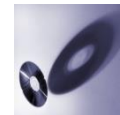
仅存在于**80386**及更高型号的微处理器中，对应两个附加的存储段，通常也用来存放数据。



2.1.2 微处理器的工作模式

实地址模式 (real-address mode)

- 也称为实模式
- 32位微处理器的工作方式与8086基本相同，但允许访问32位寄存器组
- 采用分段寻址方式，寻址1MB的存储器地址空间，不能管理和使用扩展存储器
- 寻址机制、存储器访问范围和中断控制等都与8086CPU相同
- DOS操作系统要求微处理器工作在实地址模式，计算机系统的初始化和引导程序也必须在实地址模式下运行



2.1.2 微处理器的工作模式

保护模式 (protect mode)

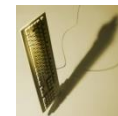
- 也称为保护的虚地址模式，支持多任务
- 保护模式提供了一系列的保护机制，为多任务操作系统的设计提供了有力的支持。
- Windows操作系统在保护模式下运行。
- 支持虚拟存储器
- 在保护模式下，CPU的32根地址线均有效，可以访问4GB的物理存储空间，以及64TB的虚拟存储空间



2.1.2 微处理器的工作模式

虚拟8086模式 (virtual 8086 mode)

- 又称V86模式
- 在V86模式下，CPU运行在保护机制中，但存储器寻址与8086相同，是一种既有保护功能又能执行8086代码的工作模式。



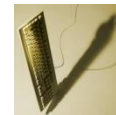
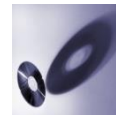
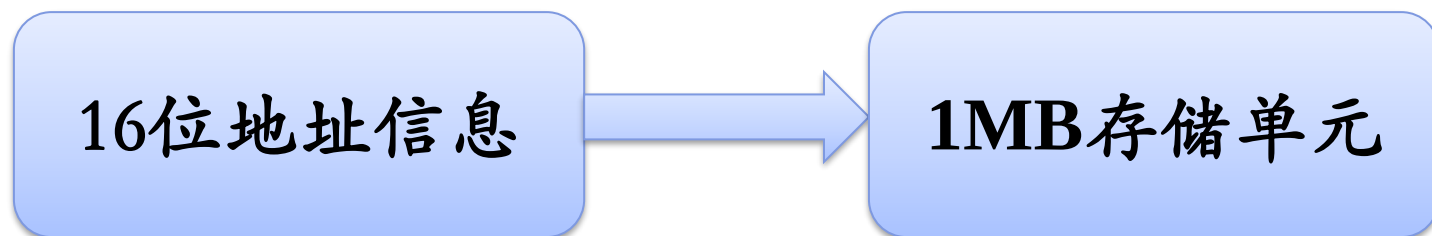
2.1.3 实地址模式下的 存储器分段寻址



为什么要分段寻址

实模式下

- 20位地址总线访问存储器可寻址的存储空间为 $2^{20}=1\text{MB}$,地址范围是00000H~FFFFFH
- 微处理器使用16位寄存器,指令给出16位地址信息



➤ 存储器的分段技术

分段原则:

1、段首地址低4位必须为全0，高16位任选

2、规定：每段的最大内存为64K，用16位二进制表示

3、每段内的任一存储单元地址可以用相对于段首地址的16位偏移量来表示。这个偏移量称当前段内偏移地址，也称有效地址EA，其范围是：0000H~FFFFH。

4、每两段之间最小相距 $2^4=16$ 个内存单元



有关的地址概念：

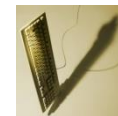
段地址：一个段的起始地址(低4位必须为全0)的高16位有效数字

有效地址：段内偏移地址，16位，简称为 EA

逻辑地址：内存单元地址的表示方法表示格式为：段地址：偏移地址。

物理地址：20位（16进制为5位）的实际地址码

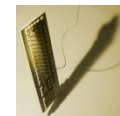
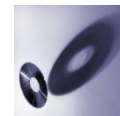
物理地址=段址址×10H+段内偏移地址



例：已知一个数存于 C018H: 1E7FH单元中，求其物理地址

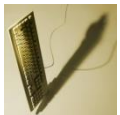
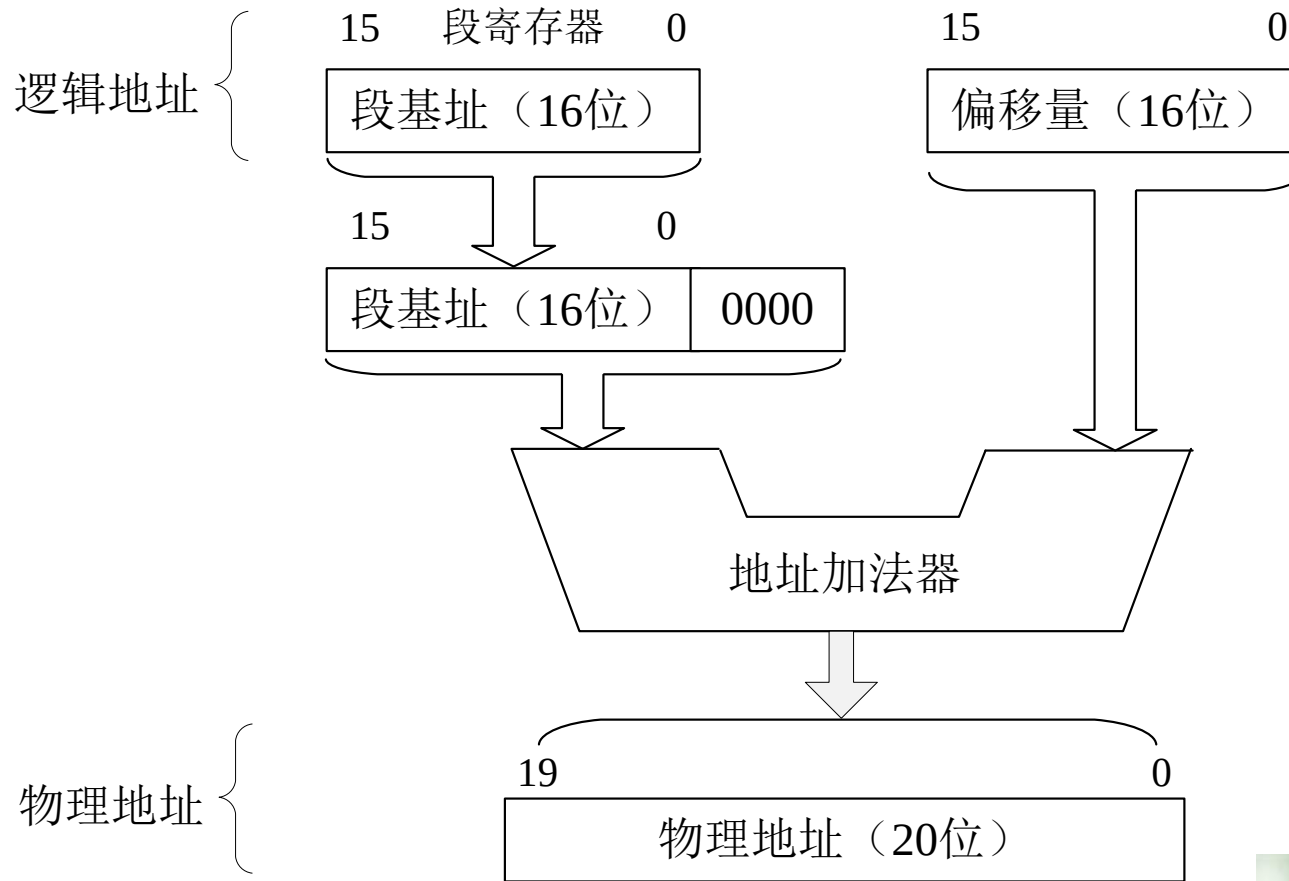
解：物理地址=C018H×10H+1E7FH

$$\begin{array}{r} \text{C 0 1 8 0 H} \\ + \text{1 E 7 F H} \\ \hline \text{C 1 F F F H} \end{array}$$



3. 实模式下物理地址的产生

物理地址 = 段基址 × 10H + 偏移地址



3. 实模式下物理地址的产生

例【2-2】 已知某存储单元的逻辑地址为1020H: 0450H, 它的物理地址为:

$$\begin{aligned}\text{物理地址} &= \text{段基址} \times 10\text{H} + \text{偏移地址} \\ &= 1020\text{H} \times 10\text{H} + 0450\text{H} \\ &= 10650\text{H}\end{aligned}$$



3. 实模式下物理地址的产生

例【2-3】 已知数据段寄存器DS的内容是30B0H，数据段中某存储单元的偏移地址为0502H，则

1) 该存储单元的物理地址

$$DS \times 10H + \text{偏移地址} = 30B00H + 0502H = 31002H$$



按信息的分段存储及分段寻址

存储器

程序区：存储程序的指令代码

数据区：存储原始数据、中间结果和最后结果

堆栈区：在堆栈区压入需保护的寄存器内容或状态等信息

1、对程序区的访问

段地址：存放在**CS**中

段内偏移地址：存放在**IP**中。

下一条要取出的指令字节物理地址： $(CS) \times 10H + (IP)$



2、对堆栈区的访问

段地址：存放在**SS**中

段内偏移地址：存放在**SP**中。

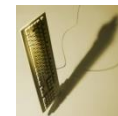
堆栈栈顶物理地址= $(SS) \times 10H + (SP)$

3、对数据区的访问

段地址：一般存放在**DS**中，可以段超越，将段地址存放在**ES**，**SS**或**CS**中

段内偏移量：**EA** 数据区某存储单元偏移地址除可由几个间址寄存器提供外，还可由各种寻址方式给出，统称为有效地址**EA**。

物理地址= $(DS) \times 10H + EA$



例：已知 $(CS)=100BH$ ， $(SS)=200AH$ ， $(DS)=300FH$
 $(IP)=0100H$ ， $(SP)=0200H$ ， $EA=0300H$

则：下一条要取指令的物理地址为：

$$100BH \times 10H + 0100H = 101B0H$$

堆栈栈顶的物理地址为：

$$200A0H + 0200H = 202A0H$$

数据区中段内偏移地址为EA的操作数物理地址为：

$$300FH \times 10H + 0300H = 303F0H$$



8086/8088的数据的存储及其寻址

● 字节数据

数据位数8位，可以存于00000H~FFFFFFH间的任意单元，对应的存储单元的地址即字节数据的地址。**CPU**存取字节数据时，只需给出对应的实际地址即可。

10H 11300H



● 字数据：存入任何相邻的两个存储单元

存放原则：高8位存在高地址单元，低8位存于低地址单元

寻址原则：低位字节地址作为字的地址

规则字：地址是偶数的字

非规则字：地址是奇数的字

一般**8086**总是把字按规则字存放，因为**8086CPU**存取一个规则字需要一个总线周期，而存取一个非规则字，则需要两个总线周期

10H	10000H
20H	10001H



- **双字数据**：4个字节，占连续4个单元

存放原则：高位字存在高地址单元，低位字存于低地址单元，每字再按字数据的存放原则存放。

寻址原则：最低位字节地址作为双字的地址

12H	10000H
34H	10001H
56H	10002H
78H	10003H



例:

00000H	32H	0AH
	59H	00H
00004H	6DH	0F3H
	⋮	⋮
90000H	21H	65H
	30H	A2H
	00H	66H
	⋮	⋮

00001H
00003H

问: 字节数据**59H**
的地址为?

00002H

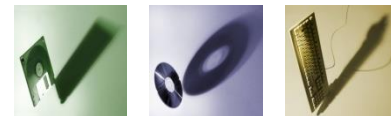
90001H
90005H

问: 字数据
590AH的地址为?

00001H

问: 存于**90000H**
的双字数据为?

A2306521H



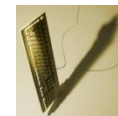
2.2 INTEL8086微处理器

【主要内容】

Intel 8086微处理器的内部结构及外部引脚

【学习目标】

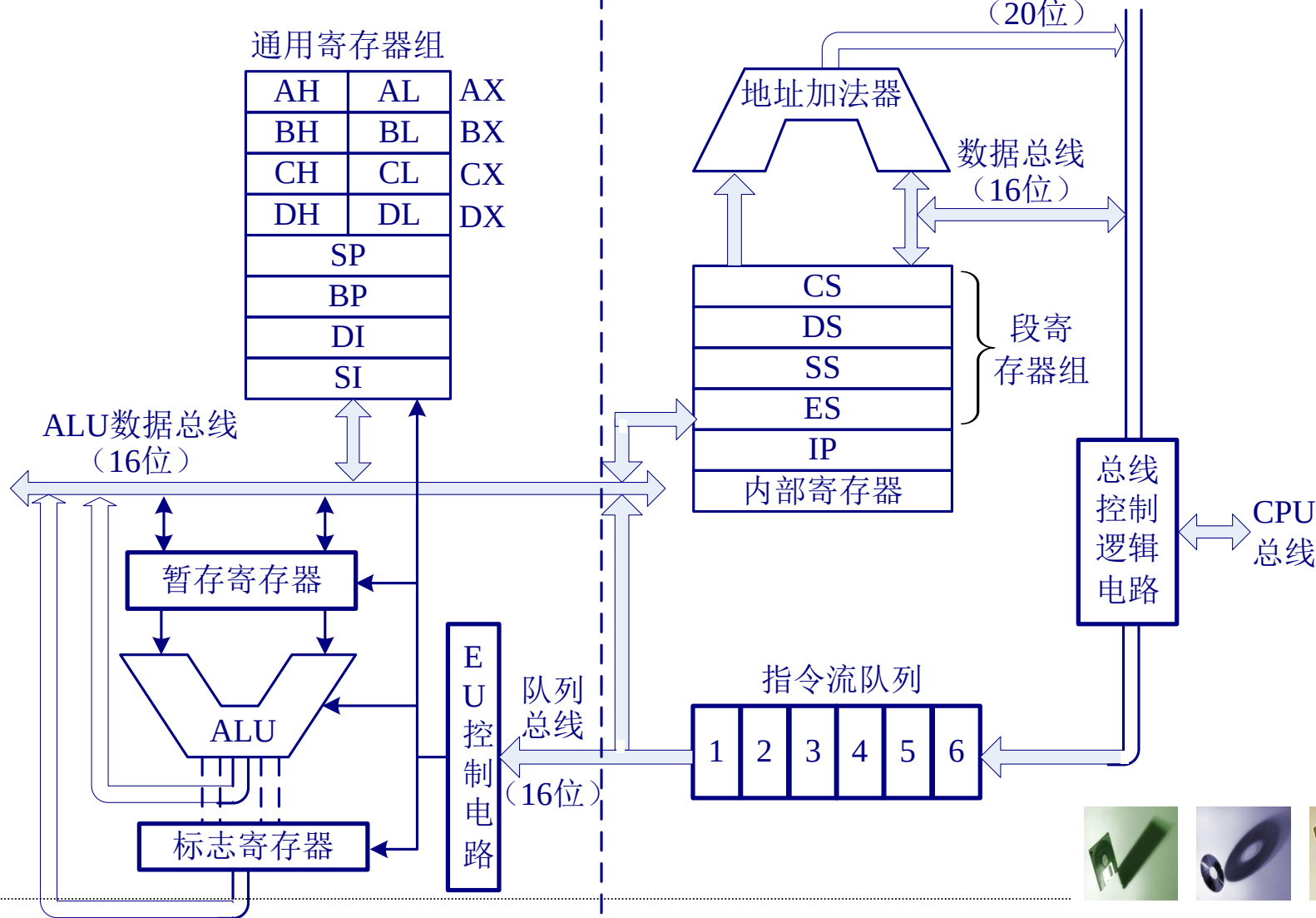
掌握8086微处理器的内部结构，执行单元和总线接口单元的结构及功能，寄存器的名称及主要作用，了解8086微处理器外部引脚的定义及功能



2.2.1 8086CPU的内部结构

Bus Interface Unit

Execute Unit 执行单元 (EU) **总线接口单元 (BIU)** 地址总线 (20位)



执行单元EU

功能:

指令的执行，从**BIU**的指令流队列中取指令，分析指令和执行指令。

组成:

(1) 算术逻辑单元 (**ALU**)

(2) **16**位标志寄存器**FLAGS**

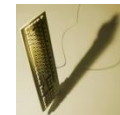
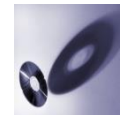
(3) 寄存器阵列

通用寄存器**AX**、**BX**、**CX**、**DX**;

专用寄存器：指针寄存器**SP**、**BP**和变址寄存器**SI**、**DI**。

(4) 数据暂存器

(5) **EU**控制电路是控制、定时状态逻辑电路，根据指令译码形成各种定时控制信号，对**EU**的各个部件实现特定的定时操作。



总线接口单元BIU

功能：负责CPU与存储器或CPU与I/O设备之间传送数据。

组成部分：

- 4个16位段寄存器
- 1个16位指令指针IP
- 长度为6个字节的指令流队列
- 地址加法器用来产生20位地址
- 总线控制电路



寄存器阵列

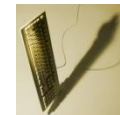
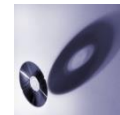
EU单元

4个16位**通用寄存器**：累加器AX，基址寄存器BX，计数器CX，数据寄存器DX，可作为16位寄存器，也可作为8位寄存器使用。

8位：AH、AL、BH、BL，CH、CL，DH、DL

2个16位**指针寄存器**：堆栈指针SP和基址指针BP

2个**变址寄存器**：源变址寄存器SI和目标变址寄存器DI



寄存器阵列

EU单元

16位状态标志寄存器FLAGS

6位状态标志：

进位标志CF、奇偶标志PF、辅助进位标志AF、
零标志ZF、 符号标志SF、 溢出标志OF

3位控制标志：

中断允许标志IF、方向标志DF和单步操作标志TF。



FLAGS标志寄存器

进位标志CF (carry flag)

反映加法运算的进位情况或减法运算的借位情况。

当加法运算的最高位有进位或减法运算的最高位有借位，CF置1；反之，CF清0

辅助进位标志AF (auxiliary carry flag)

反映加减法运算中，第3位与第4位之间的进位或借位情况。如果有进位或借位，AF置1，否则清0。

BCD码运算中，AF标志位用作十进制调整的依据。



FLAGS标志寄存器

零标志ZF(zero flag)

反映运算结果是否为零。如结果为0，则ZF置1；否则，ZF清0。

符号标志SF(signal flag)

记录运算结果的符号。

运算结果的最高位（即符号位）为1，SF=1；符号位为0，SF=0。

奇偶标志PF(parity flag)

反映运算结果1的个数是偶数还是奇数。

当1的个数为偶数，则PF=1；否则，PF=0。

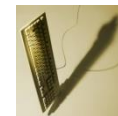


FLAGS标志寄存器

溢出标志OF(overflow flag)

反映有符号数运算结果的溢出情况。

如运算结果溢出，则OF=1；否则，OF=0。



FLAGS标志寄存器

例【2-1】 指出执行下列加法操作后各标志位的状态。

$$\begin{array}{r} 0111\ 0010\ 0101\ 1000 \\ +\ 0101\ 0011\ 0110\ 0110 \\ \hline 1100\ 0101\ 1011\ 1110 \end{array}$$

最高位无进位, **CF = 0**

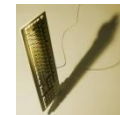
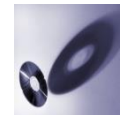
偶数个1, **PF = 1**

第3位向第4位无进位, **AF = 0**

运算结果不为0, **ZF = 0**

最高位为1, **SF = 1**

运算结果溢出, **OF = 1**。



$(-25) - (+20)$

1 1 1 0 0 1 1 1 $[-25]_{\text{补}}$

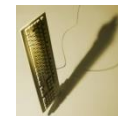
1 1 1 0 1 1 0 0 $[-20]_{\text{补}}$

1 1 0 1 0 0 1 1 $[-45]_{\text{补}}$

PF = 0 ZF = 0 OF = 0 SF = 1

最高位**有进位**，相当于**无借位CF = 0**

第3位向第4位**有进位**，相当于**无借位AF = 0**



FLAGS标志寄存器

TF, IF, DF 控制标志

TF (trap) 单步操作标志, 或陷阱标志

为程序调试的方便设置的, 用于控制单步中断

TF=1, 单步工作方式

TF=0, 正常执行程序。

IF (interrupt) 中断标志

用于控制可屏蔽的硬件中断INTR

IF=1, 允许CPU响应可屏蔽中断请求

IF=0, 禁止CPU响应可屏蔽中断请求

不影响非屏蔽中断请求 (NMI) 及内部的中断请求。



FLAGS标志寄存器

TF, IF, DF 控制标志

DF (direction) 方向标志

用于控制串操作指令中地址变化的方向

DF=1，串操作指令中的地址寄存器的内容会自动递减，操作由高地址向低地址方向进行

DF=0，则为地址递增方式，串操作由低地址向高地址方向进行。



寄存器阵列

BIU单元

指令指针寄存器IP

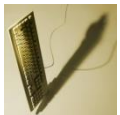
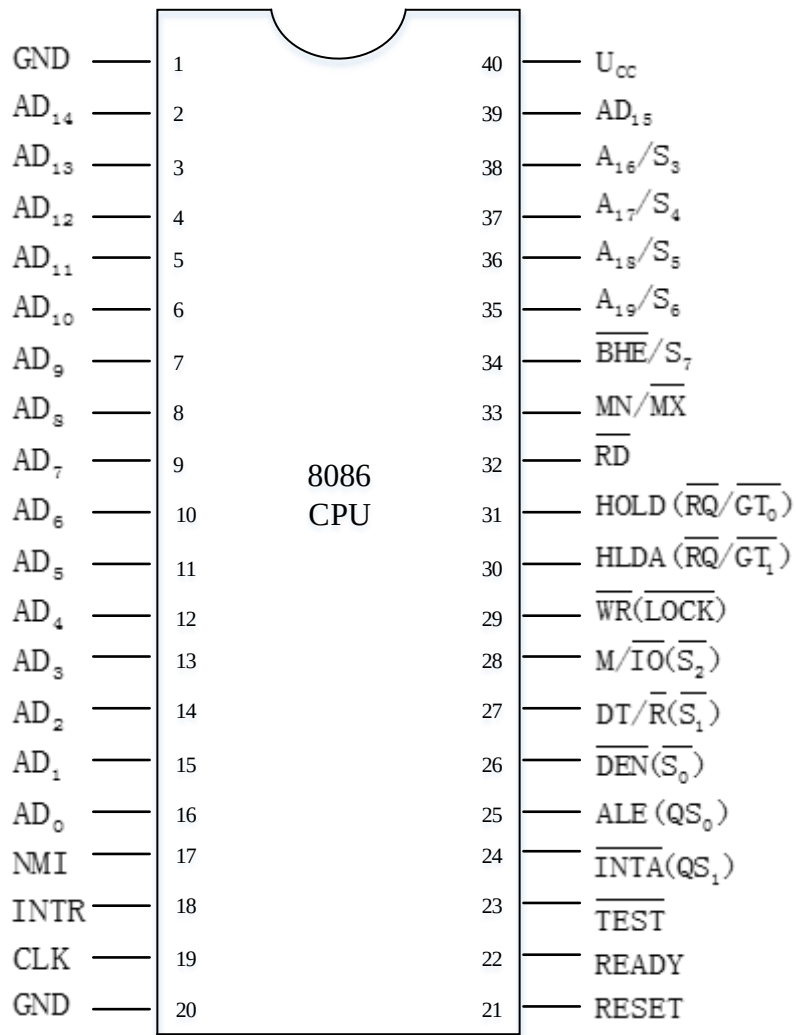
用于存放下一条将要取出的指令字节在当前代码段内的偏移地址。IP寄存器不能由程序员直接访问。

4个段寄存器CS、DS、SS和ES

分别用来存放当前代码段、当前数据段、当前堆栈段以及当前附加段的段基址。



2.2.2 8086的引脚信号和功能



2.2.2 8086的引脚信号和功能

8086 20 条地址总线，16 条数据总线

AD15—AD0 地址/数据总线 双向三态，分时复用

T1状态： **ALE=1**，地址总线**A15—A0**

其他：**ALE=0**，数据总线

DMA方式时，处于高阻状态

A19/S6—A16/S3 地址/状态复用引脚 输出，三态

T1状态： 输出地址高4位；其他状态：输出状态信息

S6 保持为 0

S5：指示IF的状态

S4、S3指示当前正在使用哪一个段寄存器

0	0	附加段寄存器
0	1	堆栈段寄存器
1	0	代码段寄存器或未用段寄存器
1	1	数据段寄存器



2.2.2 8086的引脚信号和功能

控制引脚

$\overline{\text{RD}}$

读信号，输出，低电平有效

低电平时，表示正在执行对存储器或I/O端口的读操作。

READY

准备好信号，输入，高电平有效

从CPU所寻址的存储器或I/O端口发来的回答信号

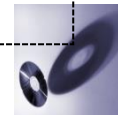
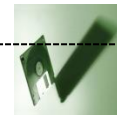
READY=1，表示外部电路已准备好，可进行一次数据传送。

INTR

可屏蔽中断请求信号，输入，高电平有效。

当INTR=1，表示外设提出了中断请求

INTR能否被响应受中断允许标志位IF的控制



2.2.2 8086的引脚信号和功能

TEST

等待测试信号，输入，低电平有效。

WAIT指令执行时，**CPU**每隔5个时钟周期测试该引脚信号为0，**CPU**将停止等待，转去执行**WAIT**指令的下一条指令；否则，继续等待，且重复测试，直到出现有效低电平为止。

NMI

非屏蔽中断请求，输入，上升沿触发。

类似于**INTR**，但**NMI**不受中断允许标志位**IF**的控制，也不能用软件屏蔽。只要此引脚上出现一个上升沿触发有效信号，**CPU**将在现行指令结束后马上响应中断，进入中断响应周期。

RESET

复位信号，输入，高电平有效。

复位信号使处理器马上结束现行操作，初始化**CPU**内部各寄存器。**8086**要求**RESET**脉冲宽度不小于4个时钟周期，接通电源时不小于50μs。系统正常运行时，**RESET**保持低电平。



2.2.2 8086的引脚信号和功能

$\overline{\text{MN}}/\overline{\text{MX}}$

最小/最大模式控制信号，输入

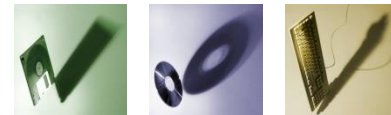
此引脚的不同设置决定了处理器的工作模式。
接+5V时，处理器工作于最小模式；
接地时，则工作于最大模式。

$\overline{\text{BHE}}/\text{S7}$

高8位数据总线允许/状态线 输出，三态

在总线周期的 T_1 状态，引脚输出低电平有效信号，表示能在高8位数据总线 $D_{15} \sim D_8$ 上传送一个字节的数据；

在 T_1 以外其他状态，此引脚输出状态信息 S_7 。此信号和 A_0 配合，指出应在存储体的哪个库（奇/偶地址库）的存储单元进行字节/字的读写操作。



2.2.2 8086的引脚信号和功能

时钟及电源引脚

CLK 系统时钟输入信号

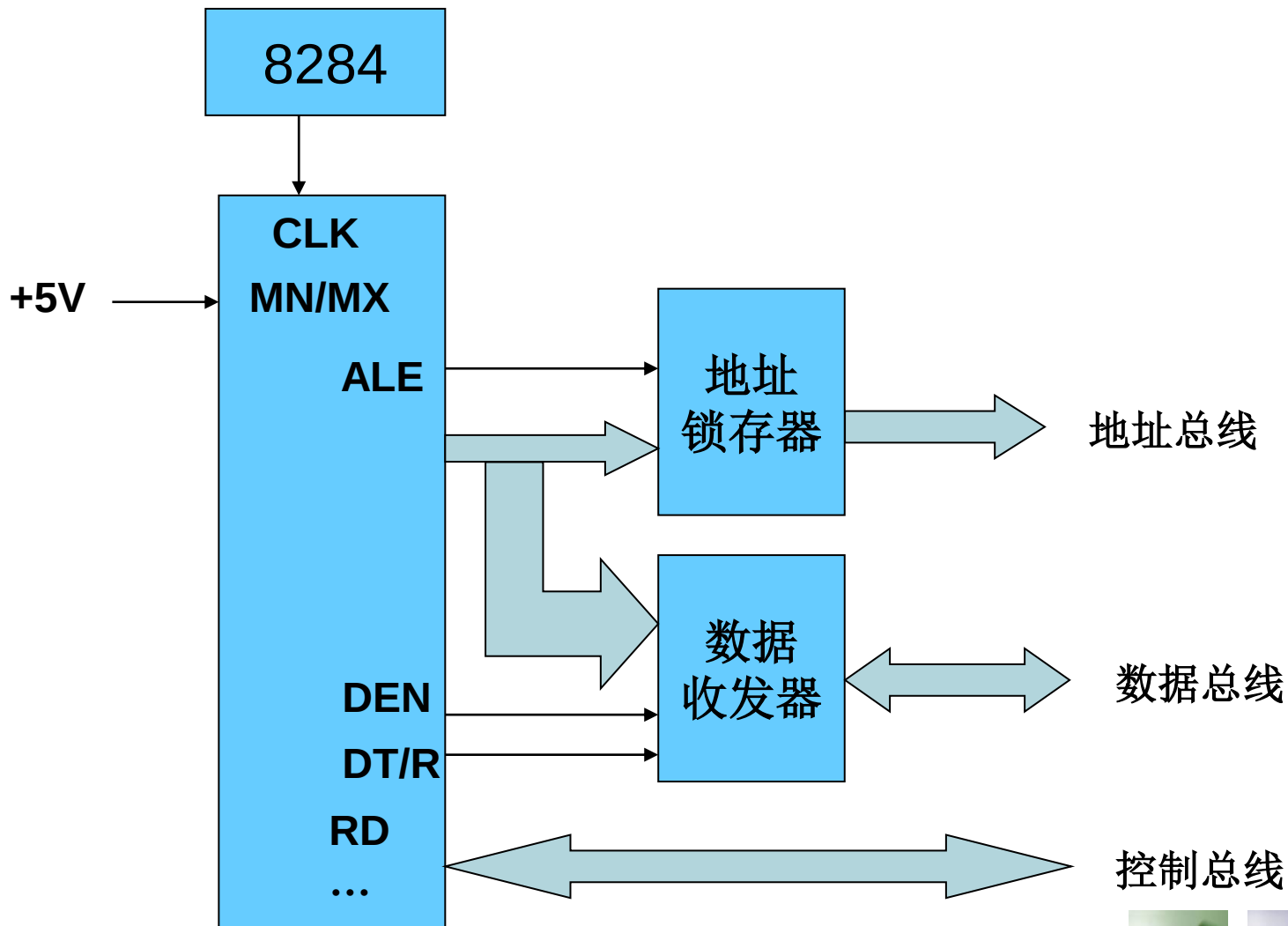
此信号通常由8284A时钟发生器提供，为处理器及总线控制器提供基本的定时脉冲。

V_{CC} +5V电源输入引脚

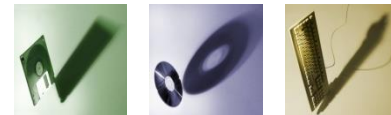
GND 接地端



最小工作模式下引脚信号



最小模式基本配置



最小工作模式下引脚信号

$\overline{\text{MN}}/\overline{\text{MX}}$ 接+5.0 V —— 最小模式

$\text{M}/\overline{\text{IO}}$ 存储器/输入输出控制信号，输出。

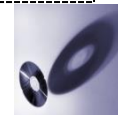
高电平，表示CPU当前与存储器进行信号传送；
低电平，表示CPU当前与I/O设备进行数据传送。

$\overline{\text{WR}}$ 写选通，输出，低电平有效

低电平：表示CPU正在对存储器或I/O端口进行写操作

$\overline{\text{INTA}}$ 中断响应信号，输出

它用于对外设的可屏蔽中断请求INTR作出响应
低电平有效



最小工作模式下引脚信号

ALE 地址锁存允许信号，输出，高电平有效

在任何一个总线周期的 T_1 状态，**ALE**有效信号随地址信息一起输出，其下降沿将地址信息锁存。

DT/ $\overline{R}$ 数据收发控制信号，输出

高电平：**CPU**发送数据

低电平：**CPU**接收数据

DEN 数据允许信号，输出

有效低电平时，表示**CPU**当前准备发送或接受一个数据



最小工作模式下引脚信号

HOLD 总线保持请求信号，输入，高电平有效

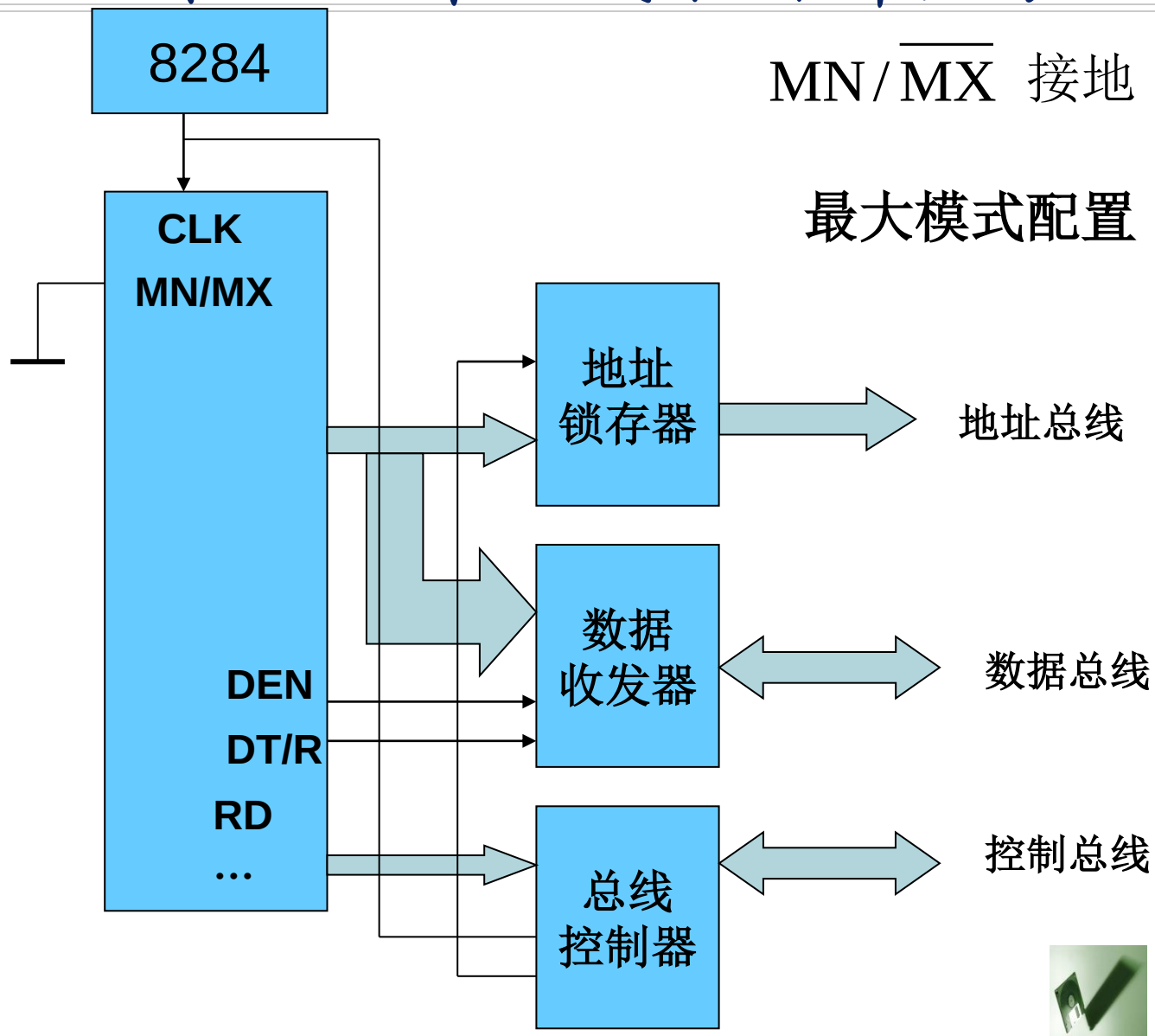
1: 请求 DMA **0: 正常执行程序**

HLDA 总线保持响应信号，输出

与**HOLD**信号配合使用.当此信号为有效高电平时，表示**CPU**已响应其他部件的请求，放弃了对总线的控制权。



最大工作模式下引脚信号



最大工作模式下引脚信号

$\overline{S2}, \overline{S1}, \text{ and } \overline{S0}$ 总线周期状态信号（输出）

提供当前总线周期中所进行的数据传输类型，由总线控制器**8288**译码，产生访问存储器和I/O端口的总线控制信号

$\overline{RO/GT1}, \overline{RO/GT0}$

总线请求信号(输入)/总线请求允许信号(输出)

可供**CPU**以外两个协处理器用来发出使用总线请求和接收**CPU**对总线请求信号的回答信号，双向

$\overline{LOCK}$ 总线封锁信号(输出)

低电平有效时，表示不允许其他部件占用总线

QS1 and QS0 指令队列状态信号(输出)

信号组合起来可反映**BIU**中指令队列的状态



本章小结

- 实模式下的存储器分段寻址，物理地址和逻辑地址
- 数据的存储及寻址
- 8086微处理器的内部结构及引脚信号

